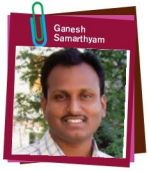

Software Engineering Involves a Lot More than Programming

In this article, the author talks about the important lessons he learnt while working on a college project—lessons that hold good even in the real world.

During my college days, a few of my classmates decided to take up a challenging project. Five of us had time to hack some code—two months of semester holidays to make use of and enough enthusiasm to take up a project that was unusual and difficult. During those days, in the late 90s, Java was getting very popular. So we decided to implement a simple compiler and virtual machine for Java as a group project.

Though we were aspiring to accomplish a lot, the chances of completing the project were slim. We had little experience writing code for a large project; we lacked coordination and did not know how to plan; we did not know what configuration management was; we knew how to use only primitive tools (like text editors) and we debugged manually by tracing the code. But there was an even more formidable challenge - none of us knew Java!

However, our objective was very clear—to implement a compiler and virtual machine for Java. The Java language specification and the virtual machine specification was (and is) available for free and is well written. A book was also available describing the internals of Java. Armed with these resources, we divided the project tasks into five parts:

- Java parser (that would use recursive descent parsing since it was the easiest to implement)
- Assembler (our Java compiler implementation would generate Java byte codes that would be 'assembled' to generate a Java '.class' file)
- Loader (the loader would dissect the '.class' file and create runtime data structures; it also passed the byte code instructions to the interpreter for execution)
- Interpreter (the stack-based interpreter would actually execute the byte code instructions that would be seen as the 'execution' of the input program to the users of the VM)
- Utilities (meant for common utility code for the compiler and VM)

The only thing we planned was to complete each of these tasks in two months (because that was the duration of our semester holidays)! To avoid 'wasting time' in discussions, we limited our communications and found our own ways to develop components independently. For instance, to implement the assembler, we would use the Java disassembler tool (i.e., *javap* tool shipped as part of JDK) and then assemble it to create '.class' files.

Independently, we struggled a lot to complete our respective modules. For instance, my task was to write a Java parser. I ran into so many problems writing the code, and spent 80 per cent or more of the time debugging the code rather than writing any useful new code.

There were some small disasters as well. We used the date or keywords like 'latest' in the folder names to keep track of versions of the folders containing source code, and did not use any version control tools. We lost source code often and had to rework them again; for instance, we had copies of the zip files in many machines or floppy drives - so, the extension 'latest' was never useful and we had to rework when we used the wrong version.

At the end of two months, we were quite happy that we had completed our tasks. We thought it was a simple task to put together our parts and get the code working, but we got a rude shock: nothing worked. The interfaces were incompatible. There were many assumptions we had made on other parts, which led to build failures (and later crashes). We thought it would take a week or two to get the final working product. But it took two more months with late night sessions and considerable reworking to get it going. We realised we spent more time integrating components than developing the independent components. The completely unused part was the utilities component - parts each of us developed used our own helper classes and methods, and the utilities component was just not used!

It was thrilling to see the final version of the compiler and interpreter working, and running as expected—starting from a simple 'Hello world' application to large programs spanning hundreds of lines of code.

We went our different ways after completing the project. But this experience gave us confidence and led to opportunities that we didn't expect at all when we started it. For instance, based on this project experience, I landed my first job as an engineer in a C++ compiler team in an MNC. Two others are now successful project managers, and the other two run their own small businesses.

Learning from experience

If I reflect upon our experience, there are many interesting things to learn from it.

Why did we divide our work into five modules? Because we were five people who had to work as independently as possible!